1. **Install Wireshark on your laptop, start a capture on your main network interface, and visit any website (like instagram.com). Stop the capture and note down the protocol names you see in the packet list.**

Ans;-

Install Wireshark and Capture Network Packets

Steps to Perform

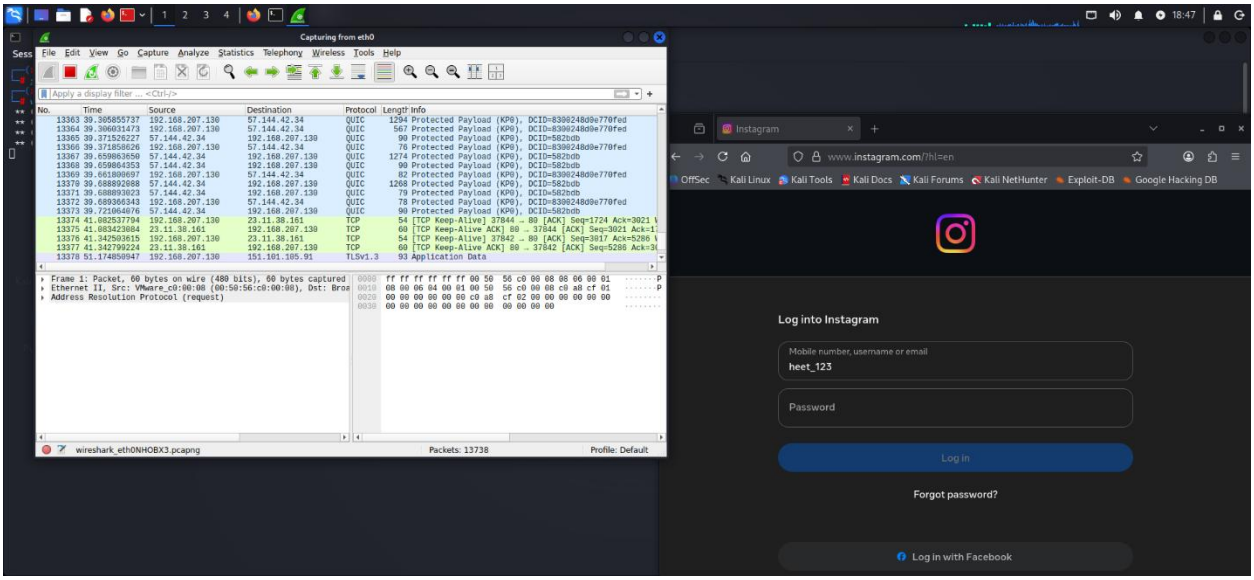
1. Open the official website: https://www.wireshark.org/download.html.
2. Download and install Wireshark on your laptop.
3. Open Wireshark.
4. Select your main network interface, such as Wi-Fi or Ethernet.
5. Double-click the interface to start capturing packets.
6. Open your browser and visit: `https://www.instagram.com`.
7. Return to Wireshark and click the red Stop button.
8. Observe the Protocol column in the packet list.

Protocols normally seen when opening a website:

Protocol	What it does	OSI Layer
DNS	Finds the IP address of the website name (e.g., instagram.com).	Application (7)
TLSv1.2 / TLSv1.3	Encrypts data between the browser and website (HTTPS).	Presentation (6)
TCP	Makes a reliable connection and delivers data in order.	Transport (4)
UDP / QUIC	Supports fast transport used by modern websites and video.	Transport (4)
HTTP	Transfers web page requests and responses.	Application (7)
ARP	Finds the MAC address of a device on the local network.	Data Link (2)
ICMPv6	Supports IPv6 network functions and diagnostics.	Network (3)
mDNS	Discovers devices and services on the local network.	Application (7)
SSDP	Helps discover devices and services on the network.	Application (7)

Note: Most websites use HTTPS, so you will commonly see TLS or QUIC traffic rather than readable HTTP data.

Protocols I saw in my capture: ARP, DNS, TCP, TLSv1.3, QUIC and HTTP (the capture had 5763 packets).


Capturing from eth0
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
No. Time Source Destination Protocol Length Info
13363 39.305855737 192.168.207.130 57.144.42.34 QUIC 1294 Protected Payload (KP0), DCID=8300248d0e770fed
13364 39.306031473 192.168.207.130 57.144.42.34 QUIC 567 Protected Payload (KP0), DCID=8300248d0e770fed
13365 39.371526227 57.144.42.34 192.168.207.130 QUIC 90 Protected Payload (KP0), DCID=582bdb
13366 39.371858626 192.168.207.130 57.144.42.34 QUIC 76 Protected Payload (KP0), DCID=8300248d0e770fed
13367 39.659863650 57.144.42.34 192.168.207.130 QUIC 1274 Protected Payload (KP0), DCID=582bdb
13368 39.659864353 57.144.42.34 192.168.207.130 QUIC 90 Protected Payload (KP0), DCID=582bdb
13369 39.661800697 192.168.207.130 57.144.42.34 QUIC 82 Protected Payload (KP0), DCID=8300248d0e770fed
13370 39.688892088 57.144.42.34 192.168.207.130 QUIC 1268 Protected Payload (KP0), DCID=582bdb
13371 39.688893023 57.144.42.34 192.168.207.130 QUIC 79 Protected Payload (KP0), DCID=582bdb
13372 39.689366343 192.168.207.130 57.144.42.34 QUIC 78 Protected Payload (KP0), DCID=8300248d0e770fed
13373 39.721064076 57.144.42.34 192.168.207.130 QUIC 90 Protected Payload (KP0), DCID=582bdb
13374 41.082537794 192.168.207.130 23.11.38.161 TCP 54 [TCP Keep-Alive] 37844 → 80 [ACK] Seq=1724 Ack=3021
13375 41.083423084 23.11.38.161 192.168.207.130 TCP 60 [TCP Keep-Alive ACK] 80 → 37844 [ACK] Seq=3021 Ack=1
13376 41.342503615 192.168.207.130 23.11.38.161 TCP 54 [TCP Keep-Alive] 37842 → 80 [ACK] Seq=3017 Ack=5286
13377 41.342709224 23.11.38.161 192.168.207.130 TCP 60 [TCP Keep-Alive ACK] 80 → 37842 [ACK] Seq=5286 Ack=3
13378 51.174850947 192.168.207.130 151.101.105.91 TLSv1.3 93 Application Data
Frame 1: Packet, 60 bytes on wire (480 bits), 60 bytes captured
Ethernet II, Src: VMware_c0:00:08 (00:50:56:c0:00:08), Dst: Broa
Address Resolution Protocol (request)
0000 ff ff ff ff ff ff 00 50 56 c0 00 08 08 06 00 01
0010 08 00 06 04 00 01 00 50 56 c0 00 08 c0 a8 cf 01
0020 00 00 00 00 00 00 c0 a8 cf 02 00 00 00 00 00 00
0030 00 00 00 00 00 00 00 00 00 00 00 00
wireshark_eth0NHOBX3.pcapng
Packets: 13738
Profile: Default
18:47
Instagram
www.instagram.com/?hl=en
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB
Log into Instagram
Mobile number, username or email
heet_123
Password
Log in
Forgot password?
Log in with Facebook

2. **Pick any 3 packets from your Wireshark capture and identify which OSI layer each packet mainly represents (for example, HTTP for Application layer, TCP for Transport layer, etc.).

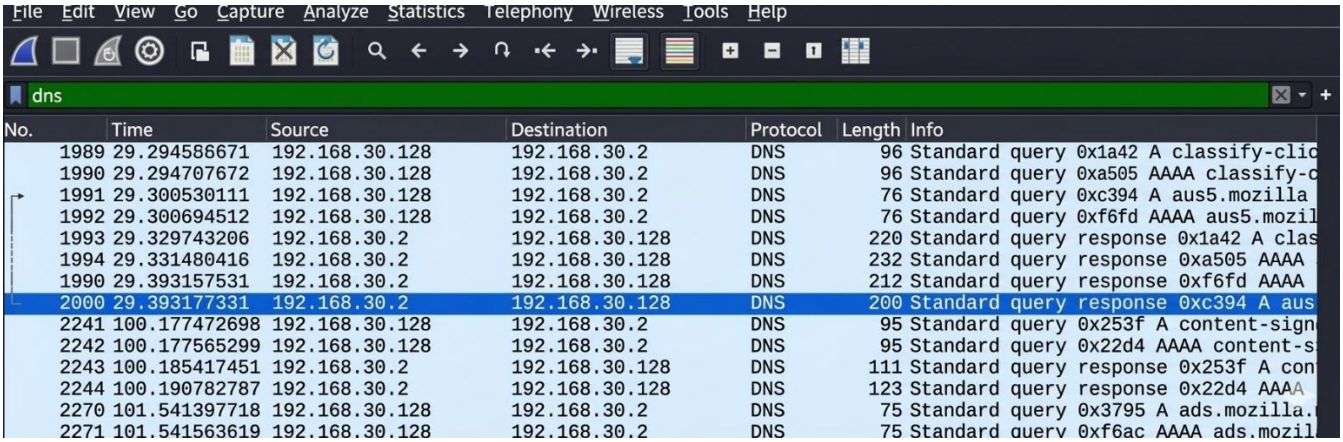
Hint: Use the Protocol column and Info details in Wireshark to help you map protocols to OSI layers.**

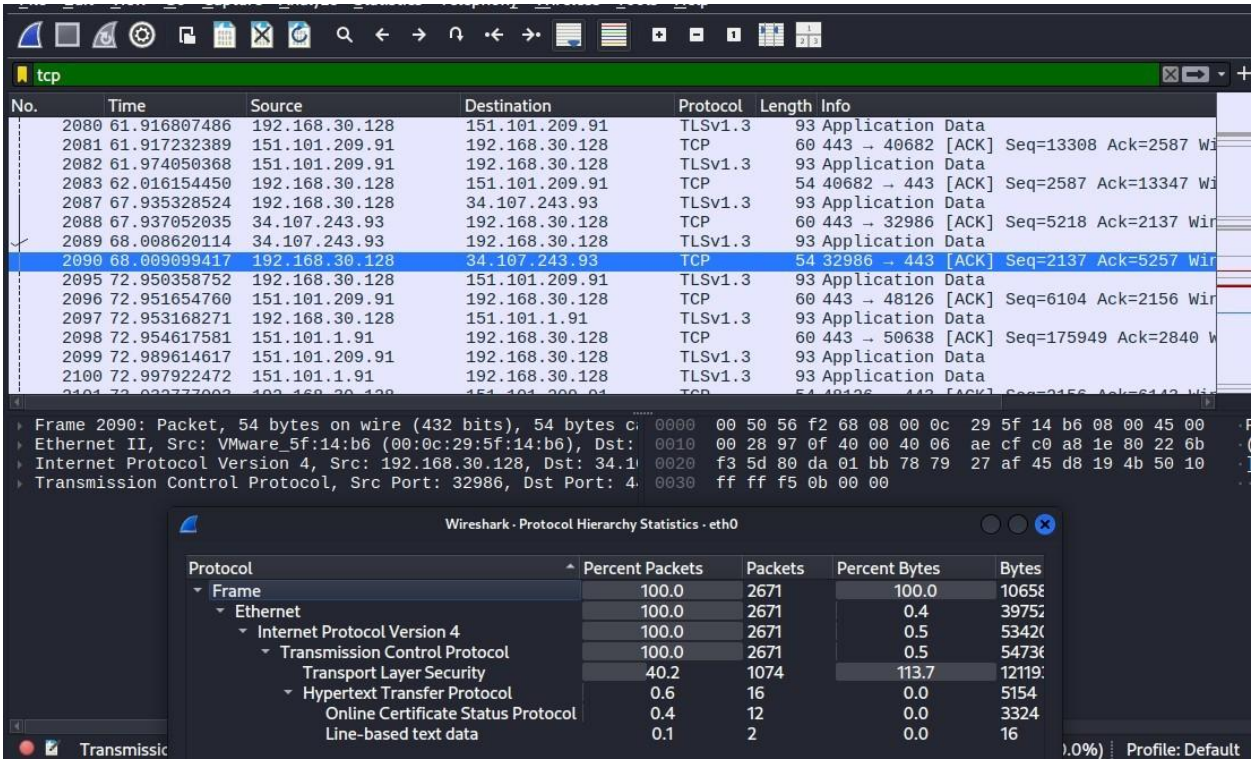
Ans:-

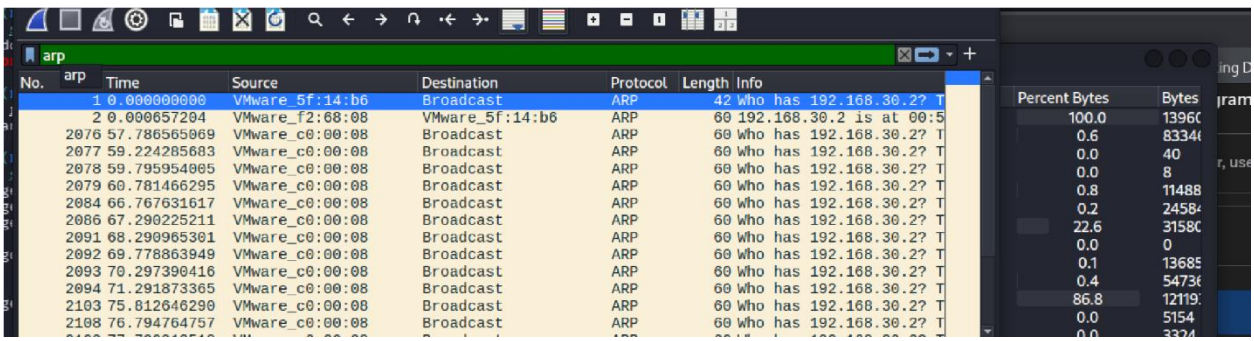
Packet no.	Protocol	Info column (typical)	OSI layer	Reason
2000	DNS	Standard query response 0xc394 A aus5.mozilla.org	Application (7)	DNS is a service used by the browser to turn a website name into an IP address.
2090	TCP	32986 > 443 [ACK] Seq=2137 Ack=5257	Transport (4)	It shows port numbers (443 = HTTPS) and the ACK that confirms data was received.
1	ARP	Who has 192.168.30.2? Tell 192.168.30.128	Data Link (2)	ARP works with MAC addresses inside the local network (Ethernet frame).

Quick Mapping Reference (from the document):

- **Application (7):** HTTP, DNS
- **Presentation (6):** TLS
- **Transport (4):** TCP, UDP
- **Network (3):** IP, ICMP
- **Data Link (2):** ARP, Ethernet / Wi-Fi frames

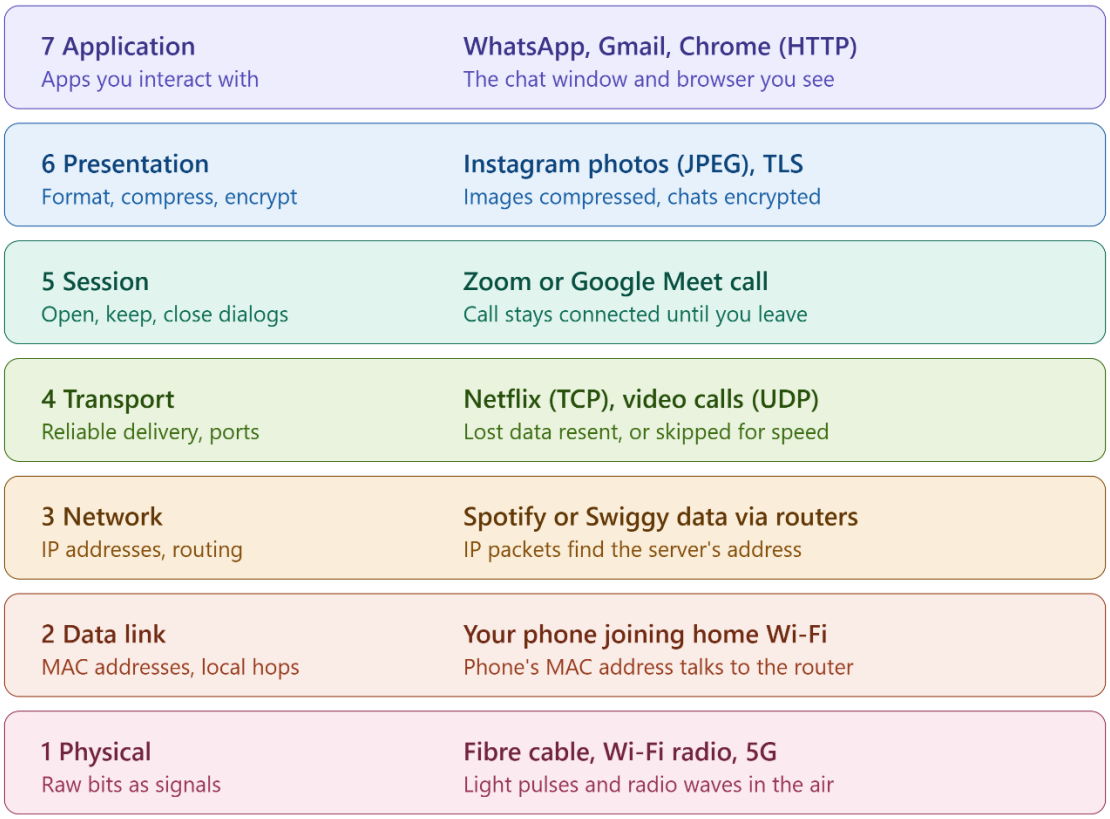



tcp
No. Time Source Destination Protocol Length Info
2080 61.916807486 192.168.30.128 151.101.209.91 TLSv1.3 93 Application Data
2081 61.917232389 151.101.209.91 192.168.30.128 TCP 60 443 → 40682 [ACK] Seq=13308 Ack=2587 Wi
2082 61.974050368 151.101.209.91 192.168.30.128 TLSv1.3 93 Application Data
2083 62.016154450 192.168.30.128 151.101.209.91 TCP 54 40682 → 443 [ACK] Seq=2587 Ack=13347 Wi
2087 67.935328524 192.168.30.128 34.107.243.93 TLSv1.3 93 Application Data
2088 67.937052035 34.107.243.93 192.168.30.128 TCP 60 443 → 32986 [ACK] Seq=5218 Ack=2137 Wir
2089 68.008620114 34.107.243.93 192.168.30.128 TLSv1.3 93 Application Data
2090 68.009099417 192.168.30.128 34.107.243.93 TCP 54 32986 → 443 [ACK] Seq=2137 Ack=5257 Wir
2095 72.950358752 192.168.30.128 151.101.209.91 TLSv1.3 93 Application Data
2096 72.951654760 151.101.209.91 192.168.30.128 TCP 60 443 → 48126 [ACK] Seq=6104 Ack=2156 Wir
2097 72.953168271 192.168.30.128 151.101.1.91 TLSv1.3 93 Application Data
2098 72.954617581 151.101.1.91 192.168.30.128 TCP 60 443 → 50638 [ACK] Seq=175949 Ack=2840 W
2099 72.989614617 151.101.209.91 192.168.30.128 TLSv1.3 93 Application Data
2100 72.997922472 151.101.1.91 192.168.30.128 TLSv1.3 93 Application Data
Frame 2090: Packet, 54 bytes on wire (432 bits), 54 bytes c
Ethernet II, Src: VMware_5f:14:b6 (00:0c:29:5f:14:b6), Dst:
Internet Protocol Version 4, Src: 192.168.30.128, Dst: 34.1
Transmission Control Protocol, Src Port: 32986, Dst Port: 4
0000 00 50 56 f2 68 08 00 0c 29 5f 14 b6 08 00 45 00
0010 00 28 97 0f 40 00 40 06 ae cf c0 a8 1e 80 22 6b
0020 f3 5d 80 da 01 bb 78 79 27 af 45 d8 19 4b 50 10
0030 ff ff f5 0b 00 00
Wireshark · Protocol Hierarchy Statistics · eth0
Protocol Percent Packets Packets Percent Bytes Bytes
Frame 100.0 2671 100.0 10658
Ethernet 100.0 2671 0.4 39752
Internet Protocol Version 4 100.0 2671 0.5 5342(
Transmission Control Protocol 100.0 2671 0.5 5473(
Transport Layer Security 40.2 1074 113.7 12119
Hypertext Transfer Protocol 0.6 16 0.0 5154
Online Certificate Status Protocol 0.4 12 0.0 3324
Line-based text data 0.1 2 0.0 16
Transmissic
Profile: Default


arp
No. Time Source Destination Protocol Length Info
1 0.000000000 VMware_5f:14:b6 Broadcast ARP 42 Who has 192.168.30.2? T
2 0.000657204 VMware_f2:68:08 VMware_5f:14:b6 ARP 60 192.168.30.2 is at 00:5
2076 57.786565969 VMware_c0:00:08 Broadcast ARP 60 Who has 192.168.30.2? T
2077 59.224285683 VMware_c0:00:08 Broadcast ARP 60 Who has 192.168.30.2? T
2078 59.795954005 VMware_c0:00:08 Broadcast ARP 60 Who has 192.168.30.2? T
2079 60.781466295 VMware_c0:00:08 Broadcast ARP 60 Who has 192.168.30.2? T
2084 66.767631617 VMware_c0:00:08 Broadcast ARP 60 Who has 192.168.30.2? T
2086 67.290225211 VMware_c0:00:08 Broadcast ARP 60 Who has 192.168.30.2? T
2091 68.290965301 VMware_c0:00:08 Broadcast ARP 60 Who has 192.168.30.2? T
2092 69.778863949 VMware_c0:00:08 Broadcast ARP 60 Who has 192.168.30.2? T
2093 70.297390416 VMware_c0:00:08 Broadcast ARP 60 Who has 192.168.30.2? T
2094 71.291873365 VMware_c0:00:08 Broadcast ARP 60 Who has 192.168.30.2? T
2103 75.812646290 VMware_c0:00:08 Broadcast ARP 60 Who has 192.168.30.2? T
2108 76.794764757 VMware_c0:00:08 Broadcast ARP 60 Who has 192.168.30.2? T
Percent Bytes Bytes
100.0 13960
0.6 8334(
0.0 40
0.0 8
0.8 11488
0.2 2458
22.6 3158(
0.0 0
0.1 1368
0.4 5473(
86.8 12119
0.0 5154

3. **Create a colorful diagram of the OSI Model using any drawing tool (MS Paint, Canva, or even paper and upload a photo). For each layer, add a real-world example from apps you use daily (e.g., WhatsApp uses the Application layer, Instagram images use the Presentation layer, etc.).**

Ans:-



Sender: data moves down layers 7 to 1. Receiver: up from 1 to 7.

What happens when you send a WhatsApp photo?

- **Layer 7:** You tap send in WhatsApp.
- **Layer 6:** The photo is compressed to JPEG and encrypted.
- **Layer 5:** A session is opened and kept alive with the server.
- **Layer 4:** The data is split into segments, and TCP makes sure none are lost.
- **Layer 3:** Each segment gets an IP address and is routed across the internet.
- **Layer 2:** Your phone passes frames to the Wi-Fi router using MAC addresses.
- **Layer 1:** The bits travel as radio waves and then as light pulses in fibre cable.

The receiver's device then reverses these steps, from layer 1 back up to layer 7.

4. **Explain how sending a message on WhatsApp travels through at least 4 layers of the OSI Model, describing what happens at each layer in your own words.

Hint: Think about how your message is typed, packaged, sent over the internet, and received.**

Ans:-

* How a WhatsApp message travels through the OSI Model

Say I type "Hi!" and press send. Before it reaches my friend's phone, it goes down through the layers on my device, across the network, and back up through the layers on theirs.

* Sending (top to bottom)

- **Layer 7, Application:** I open WhatsApp, type "Hi!", and tap send. WhatsApp is the app that takes my words and turns them into a message to deliver. Everything I see and touch happens here.

- **Layer 6, Presentation:** The message is changed into a form that is safe and ready to travel. The text is converted into data (bits), and WhatsApp end-to-end encryption scrambles it so only my friend can read it. If I had attached a photo, it would also be compressed here (for example, into JPEG).
- **Layer 5, Session:** WhatsApp opens and maintains a connection between my phone and WhatsApp servers. This is why I stay "connected" and can keep chatting without logging in again for every message. When I close the app or lose the connection, the session ends.
- **Layer 4, Transport:** The encrypted message is broken into small pieces called segments. Each piece is numbered and tagged with a port number so it reaches the right app. TCP checks that every piece arrives. If one goes missing, it is sent again.
- **Layer 3, Network:** Each segment is put into a packet with my IP address and the destination's IP address. Routers on the internet read these addresses and pick the best path, hopping from one router to the next until the packet reaches WhatsApp's server and then my friend's network.
- **Layer 2, Data link:** On each local hop, such as my phone to my Wi-Fi router, the packet is wrapped into a frame carrying MAC addresses. A MAC address is the unique hardware ID of a device, so it tells the next device on that local link who the frame is meant for.
- **Layer 1, Physical:** Finally, the frame is turned into actual signals: radio waves from my phone to the router, then electrical or light pulses through cables and fibre across the internet.

Receiving (bottom to top)

My friend's phone picks up the signals at Layer 1 and rebuilds the frames at Layer 2. Layer 3 confirms the packets are addressed to it. Layer 4 puts the numbered pieces back in order and checks that none are missing. Layer 5 keeps the session going. Layer 6 decrypts the message. Finally, at Layer 7, WhatsApp shows "Hi!" on my friend's screen.